Formal Verification of Number-Theoretic Transform in Lean

Salih Erdem Koçak, Doran Pamukçu

Advisor: Doğan Ulus   Department of Computer Engineering, Boğaziçi University

Motivation

Polynomial multiplication is a core component of many cryptographic protocols, especially proof systems. Evaluation and interpolation routines, polynomial commitments, and quotient-polynomial constructions all multiply polynomials repeatedly. With the usual schoolbook algorithm, this is quadratic work. The NTT avoids direct convolution by evaluating both inputs on a root-of-unity domain, multiplying the values pointwise, and then transforming back. Our Lean proof connects this executable pipeline to the algebraic specification, showing that the fast path is correct.

Method	Main Operation	Work
Naive	sum coefficient pairs	$O(n^2)$
NTT	transform, pointwise multiply, transform back	$O(n \log n)$

Implementation Surface

parameters — **Domain**: size, root of unity, inverse root

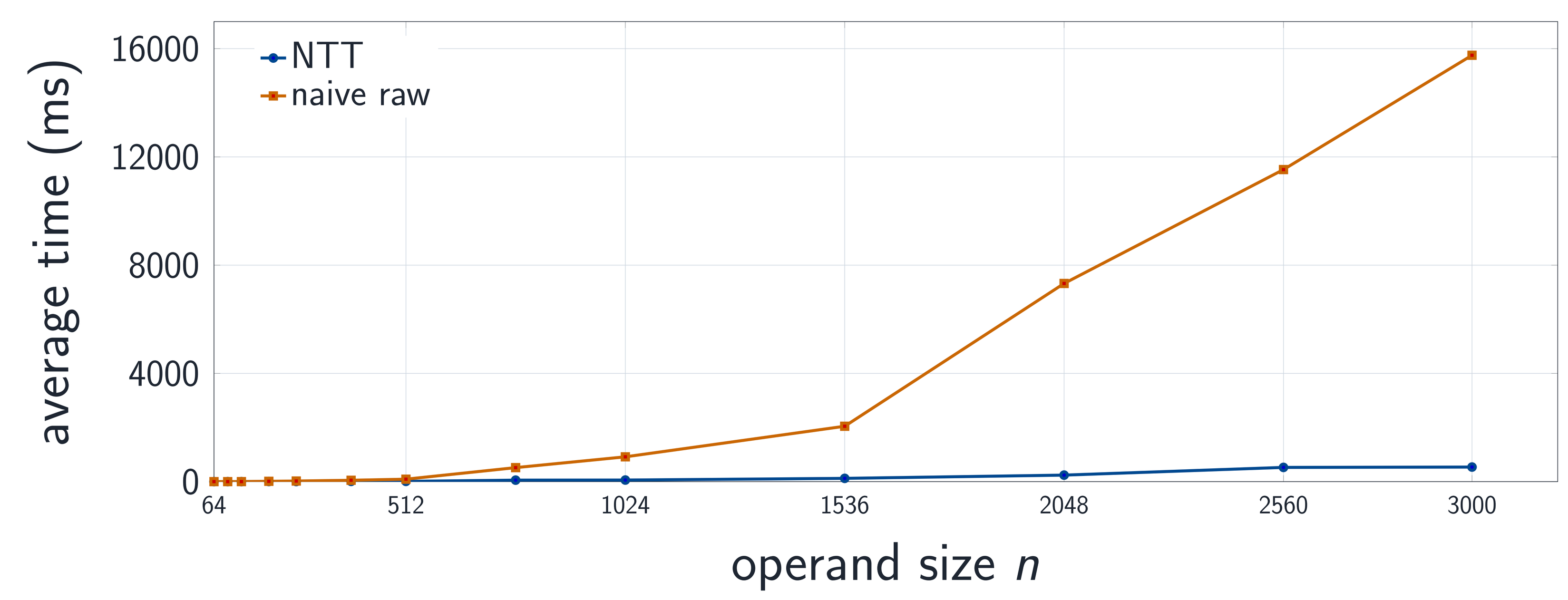
algorithm — **Forward / Inverse Transform**: iterative radix-2 butterfly implementation

pipeline — **FastMul.Raw**: forward transform, pointwise multiplication, inverse transform

public API — **FastMul**: canonical `CPolynomial` result

The public operation is small; most of the work is in connecting executable array code to mathematical transform specifications and then to polynomial multiplication.

Benchmark Behavior



Manual KoalaBear benchmark sweep from `tests/CompPolyTests/Univariate/NTT/Benchmark.lean`. Times are illustrative and machine-dependent.

NTT Multiplication

coefficient inputs — **evaluation domain** — **coefficient output**

$p(x) = a_0 + a_1 x + \cdots$ → **NTT** → $\widehat{p} = [p(\omega^i)]_{i<n}$

$q(x) = b_0 + b_1 x + \cdots$ → **NTT** → $\widehat{q} = [q(\omega^i)]_{i<n}$

pointwise product: $\widehat{c} = \widehat{p} \odot \widehat{q}$, $\widehat{c}_i = \widehat{p}_i \widehat{q}_i$ → **inverse NTT** → $c(x) = p(x)q(x)$, $c_k = \sum_{i+j=k} a_i b_j$

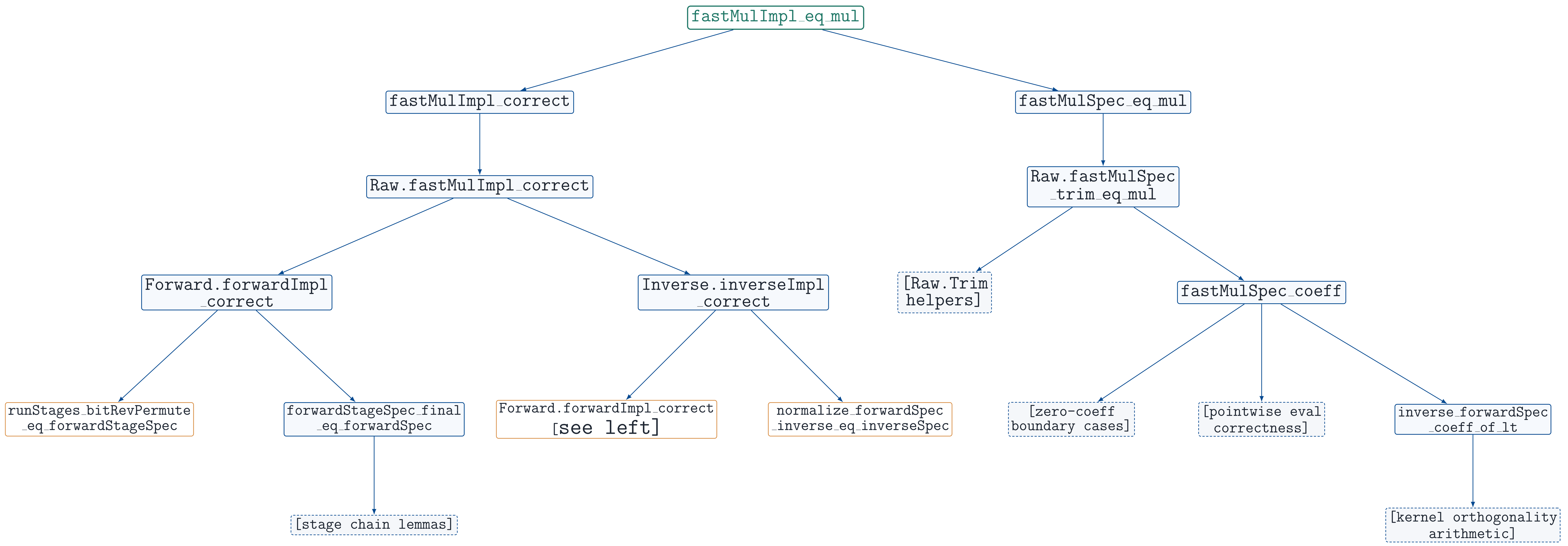
The transform evaluates at powers of a root of unity. Multiplication becomes pointwise in the evaluation domain, then the inverse transform returns the coefficient representation.

Main Theorem

```
theorem fastMulImpl_eq_mul
    (D : Domain R)
    (p q : CPolynomial R)
    (hfit : Domain.fits D p.val q.val) :
    fastMulImpl D p q = p * q
```

This is the public correctness statement for the multiplication implementation. The precondition `Domain.fits D p.val q.val` says that the selected NTT domain is large enough for the ordinary convolution, so the transform pipeline has no cyclic wraparound.

Proof Architecture



References

1. Trieu, A., "Formally Verified Number-Theoretic Transform." *IACR Communications in Cryptology*, vol. 2, no. 4, Jan. 2026. DOI: 10.62056/ahbn-4tw9.
2. Capretta, V., "Certified Fast Fourier Transform." *Theorem Proving in Higher Order Logics*, TPHOLs 2001. Springer, 2001.
3. Ballarin, C., "Fast Fourier Transform." *Archive of Formal Proofs*, Oct. 2005. isa-afp.org/entries/FFT.html.
4. Truth Research ZK, "AMO-Lean", 2026. github.com/lambdaclass/amo-lean.

Code and Repository

Merged PR #174

CompPoly Repository